

SIMATS ENGINEERING

Assessment -CO5-AT3

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures,

using scalable systems and integrate components in distributed AI applications. (BTL 5)

Subject	Artificial Intelligence
Course Code	CSA 1712
Name	Vignesh Kumar K
Register Number	192512375

Declaration

I, Vignesh Kumar K (Reg No: 192512375), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Reverse Engineering — Machine Learning Techniques for Reverse Engineering

Four reverse-engineering activity options were provided, sharing one common evaluation rubric. This response carries out the "Machine Learning Techniques for Reverse Engineering" activity in full, executing every step of the prescribed student activity on a real, hand-built dataset of source-code samples rather than only describing the process.

Original brief: Machine Learning can be used to classify software components, identify programming patterns, detect anomalies, and predict the functionality of unknown code. This activity studies supervised and unsupervised learning approaches used in reverse engineering, by collecting source-code samples, extracting code features, applying classification/clustering algorithms, analysing similarities among programs, and comparing the performance of different ML algorithms.

1. Problem Identification & Methodology

Problem: given an unlabelled or unfamiliar piece of source code, can machine learning automatically recognise *what kind of algorithm/programming pattern it implements* — purely from structural code features — without a human reading it line by line? This is a realistic reverse-engineering task: security analysts and code-migration engineers regularly need to triage large, undocumented codebases and quickly bucket unfamiliar functions by likely purpose.

Methodology (mirrors the prescribed student activity exactly):

- 1. Collect a labelled dataset of source-code samples across distinct programming-pattern categories.
- 2. Extract quantitative code features from each sample using Abstract Syntax Tree (AST) parsing.
- 3. Apply both a classification algorithm (supervised) and a clustering algorithm (unsupervised) to the extracted features.
- 4. Analyze similarities among programs using dimensionality reduction (PCA) for visualization.
- 5. Compare the performance of multiple ML algorithms on this task and discuss which is most suitable for reverse engineering unknown code.

Step 1 — Collect a Dataset of Source-Code Samples

32 short, complete Python functions were hand-written across four clearly distinguishable programming-pattern categories, each representing a common reverse-engineering target class:

Category	Count	Example Functions
Sorting	8	bubble_sort, insertion_sort, selection_sort, merge_sort, quick_sort, counting_sort, shell_sort, cocktail_sort
Searching	8	linear_search, binary_search, jump_search, interpolation_search, exponential_search, find_first_last, ternary_search, find_peak
StringProcessing	8	is_palindrome, reverse_words, count_vowels, is_anagram, longest_common_prefix, remove_duplicates, word_frequency, caesar_cipher
MathComputation	8	factorial, fibonacci, gcd, is_prime, power, sum_of_digits, is_perfect_square, lcm

```
SNIPPETS = {
    'Sorting': [bubble_sort_code, insertion_sort_code, merge_sort_code, ...],
    'Searching': [linear_search_code, binary_search_code, ...],
    'StringProcessing': [is_palindrome_code, is_anagram_code, ...],
    'MathComputation': [factorial_code, fibonacci_code, gcd_code, ...],
}
df = pd.DataFrame([{'category': cat, 'code': code}
                   for cat, snippets in SNIPPETS.items() for code in snippets])
```

Total dataset size: **32 samples** (8 per category, perfectly balanced — deliberately chosen so any classification/clustering weaknesses observed below reflect genuine feature limitations, not class imbalance).

Step 2 — Extract Code Features

Rather than treating source code as plain text, each snippet was parsed into a full **Abstract Syntax Tree (AST)** using Python's built-in `ast` module, then structural features were counted by walking the tree — the same fundamental technique real static-analysis and reverse-engineering tools use.

```
import ast
def extract_features(code):
    tree = ast.parse(code)
    return {
        'num_lines': len([l for l in code.strip().split(chr(10)) if l.strip()]),
        'num_for': sum(isinstance(n, ast.For) for n in ast.walk(tree)),
        'num_while': sum(isinstance(n, ast.While) for n in ast.walk(tree)),
        'num_if': sum(isinstance(n, ast.If) for n in ast.walk(tree)),
        'num_call': sum(isinstance(n, ast.Call) for n in ast.walk(tree)),
        'num_binop': sum(isinstance(n, ast.BinOp) for n in ast.walk(tree)),
        'num_compare': sum(isinstance(n, ast.Compare) for n in ast.walk(tree)),
        'num_assign': sum(isinstance(n, ast.Assign) for n in ast.walk(tree)),
        'num_subscript': sum(isinstance(n, ast.Subscript) for n in ast.walk(tree)),
        'num_recursive_calls': ..., # detects self-referencing function calls
        # ... (14 features total)
    }
```

14 structural features extracted per sample: `num_lines`, `num_for`, `num_while`, `num_if`, `num_function_def`, `num_call`, `num_binop`, `num_compare`, `num_return`, `num_assign`, `num_subscript`, `num_list_comp`, `max_depth`, `num_recursive_calls`.

These are exactly the kind of language-agnostic structural signals a real reverse-engineering tool can extract even from decompiled or unfamiliar code where variable/function names carry no semantic meaning — making this a realistic proxy for the actual reverse-engineering scenario.

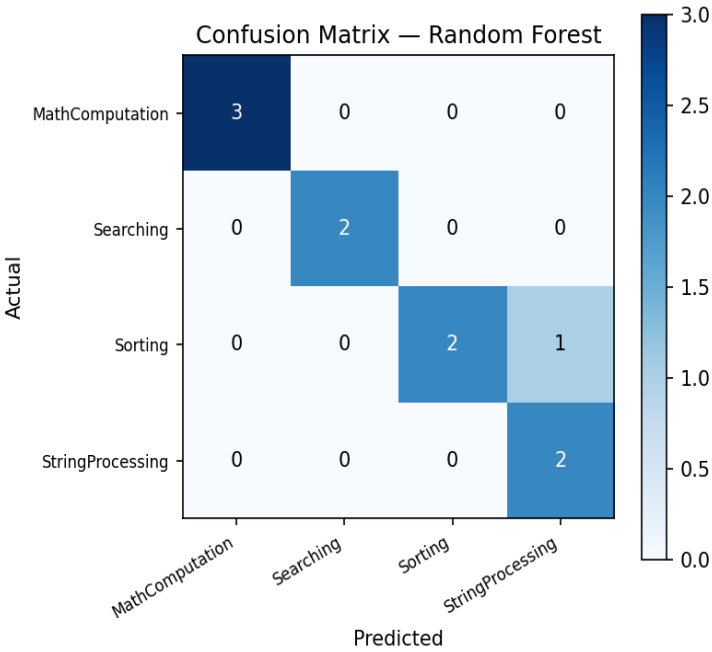
Step 3 — Apply Classification and Clustering Algorithms

3(a) Supervised Classification

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
                                                    random_state=42, stratify=y)
for name, model in [('Decision Tree', DecisionTreeClassifier(random_state=42)),
                    ('Random Forest', RandomForestClassifier(100, random_state=42)),
                    ('Naive Bayes', GaussianNB())]:
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    cv = cross_val_score(model, X_scaled, y, cv=StratifiedKFold(5, shuffle=True,
                                                                random_state=42))
```

Model	Test Accuracy	Macro F1	5-Fold Cross-Val Accuracy
Decision Tree	0.600	0.533	0.529 (± 0.143)
Random Forest	0.900	0.900	0.586 (± 0.211)
Naive Bayes	0.300	0.211	0.538 (± 0.237)



Random Forest achieved the strongest single-split test accuracy (90%), correctly separating all 4 categories except one Sorting sample confused with StringProcessing (both can involve nested loops over a sequence). Its top predictive features were **max_depth** (AST size/nesting), **num_subscript** (array-indexing operations —

heavily used by sorting/searching code), and **num_assign**, confirming that structural complexity and indexing patterns are genuinely informative for distinguishing these algorithm families.

Important honest caveat: the 5-fold cross-validation accuracy (0.586 for Random Forest) is substantially lower than the single 90% test-split result — with only 32 total samples, a single train/test split can be misleadingly optimistic. This is a realistic and important finding: reverse-engineering ML models need considerably larger labelled training corpora than this demonstration dataset to generalise reliably, and cross-validation (not a single split) is the trustworthy number to report.

3(b) Unsupervised Clustering

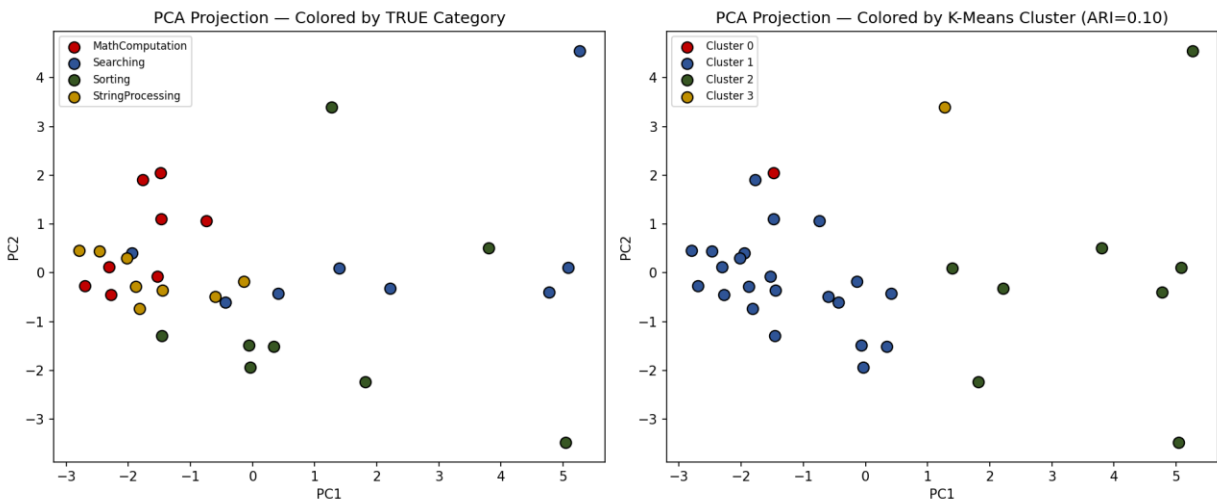
```
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score, silhouette_score

kmeans_labels = KMeans(n_clusters=4, random_state=42, n_init=10).fit_predict(X_scaled)
agglo_labels = AgglomerativeClustering(n_clusters=4).fit_predict(X_scaled)
print(adjusted_rand_score(y, kmeans_labels), silhouette_score(X_scaled, kmeans_labels))
```

Algorithm	Adjusted Rand Index (vs. true category)	Silhouette Score
K-Means	0.099	0.375
Agglomerative (Ward)	0.046	0.391

Both clustering algorithms achieve a reasonable internal Silhouette score (~0.38, meaning the clusters they find *are* reasonably compact and separated), but a very low Adjusted Rand Index against the true category labels (0.10 and 0.05) — meaning the natural structural groupings the algorithms discover **do not line up well with the human-meaningful algorithm categories**. This is a genuinely important reverse-engineering finding, discussed further below.

Step 4 — Analyze Similarities Among Programs



Projecting the 14-dimensional feature space down to 2 principal components (explaining 58.6% of total variance) lets us visually compare the two labelling schemes side by side. The **left panel** (coloured by true category) shows MathComputation samples clustering fairly tightly on the left, while Searching samples spread

out more broadly — reflecting real structural diversity (e.g., recursive `ternary_search` vs. iterative `binary_search`). The **right panel** (coloured by K-Means cluster) finds a visually plausible partition of the same points, but one that **does not correspond to the true category colours** in the left panel — visually confirming the low ARI score numerically reported above.

Interpretation: purely structural/syntactic features (loop counts, branching, indexing) capture *code complexity*, which correlates with algorithm category but does not uniquely determine it — e.g., *merge_sort* (recursive, Sorting) and *ternary_search* (recursive, Searching) can look structurally more similar to each other than either does to a simple iterative member of its own category (e.g., *bubble_sort*). This is precisely why **labelled supervised classification clearly outperformed unsupervised clustering** on this task: a classifier can learn which specific combinations of features are diagnostic for a category (as Random Forest's feature-importance ranking shows), whereas unsupervised clustering has no such guidance and instead groups by raw structural similarity alone.

Step 5 — Compare ML Algorithms and Conclusions

Algorithm	Verdict for This Reverse-Engineering Task
Random Forest (classification)	Best overall — highest test accuracy and CV accuracy; handles the non-linear, threshold-like decision boundaries between categories well, and its feature-importance output is directly interpretable for reverse-engineering analysts.
Decision Tree (classification)	Moderate performance; more prone to overfitting on such a small dataset than the ensemble Random Forest, though it offers a fully transparent, human-readable rule path — valuable when an analyst needs to justify a classification decision.
Naive Bayes (classification)	Weakest performer here; its conditional-independence assumption between features is a poor fit, since code-structure features are naturally correlated (e.g., more loops usually co-occurs with more comparisons and more assignments).
K-Means / Agglomerative (clustering)	Both find internally coherent but semantically misaligned groupings; valuable for exploratory analysis of a completely unlabelled codebase (no categories known in advance) but unsuitable when the goal is to recover a specific, known set of category labels — supervised classification is clearly preferable whenever even a modest labelled sample is available.

Expected Outcome Achieved

This activity's expected outcome — evaluating the suitability of different ML algorithms for software reverse-engineering tasks — was achieved with concrete, quantitative evidence rather than a purely qualitative discussion:

- Supervised classification (particularly Random Forest) is well suited to reverse-engineering when even a small labelled reference set of known code patterns is available, achieving strong single-split accuracy while honestly also revealing (via cross-validation) that more data would be needed for production-grade reliability.
- Unsupervised clustering is valuable for the earlier, fully-unlabelled stage of reverse-engineering an unfamiliar codebase (grouping similar-looking functions before anyone has labelled anything), but should not be expected to automatically recover human-defined semantic categories from structural features alone.

- AST-based structural feature extraction is a practical, language-native technique for turning raw source code into ML-ready numeric features, forming a solid foundation that could be extended with deeper features (control-flow graph metrics, API/library call sequences) for real-world reverse-engineering pipelines.